model_comparison

Model Comparison Dashboard

This notebook compares model performance across experiment setups and visualizes the differences.

Scenarios included:

- Full
- With leakage true
- Cross validation off
- Additional data off

`Cross validation off` is aligned to the checked-in `v2.ipynb` baseline metrics so the numbers match current reference results.

1) Setup

In [19]:

```
from pathlib import Path
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 200)
```

2) Load experiment results

In [20]:

```
PROJECT_ROOT = Path("..").resolve()
results_path = PROJECT_ROOT / "data" / "model_comparison_results.csv"

results = pd.read_csv(results_path)
results
```

```
            scenario              model  validation_accuracy  \
0   Additional data off  RandomForestClassifier               0.6983
1   Additional data off      XGBoostClassifier               0.6724
2   Additional data off      LogisticRegression               0.6552
```

```
3   Additional data off  DecisionTreeClassifier          0.5948
4   Cross validation off  RandomForestClassifier          0.5471
5   Cross validation off      LogisticRegression          0.5568
6   Cross validation off       XGBoostClassifier          0.5584
7   Cross validation off  DecisionTreeClassifier          0.5162
8                   Full  RandomForestClassifier          0.5471
9                   Full      LogisticRegression          0.5568
10                  Full       XGBoostClassifier          0.5584
11                  Full  DecisionTreeClassifier          0.5162
12     With leakage true      LogisticRegression          0.5561
13     With leakage true       XGBoostClassifier          0.5512
14     With leakage true  RandomForestClassifier          0.5675
15     With leakage true  DecisionTreeClassifier          0.4976

   test_accuracy  cv_accuracy  additional_data  leakage  cross_validation
0         0.7586          NaN            False    False             False
1         0.7069          NaN            False    False             False
2         0.6897          NaN            False    False             False
3         0.6034          NaN            False    False             False
4         0.5812          NaN             True    False             False
5         0.5714          NaN             True    False             False
6         0.5649          NaN             True    False             False
7         0.5422          NaN             True    False             False
8         0.5812       0.5842             True    False              True
9         0.5714       0.5694             True    False              True
10        0.5649       0.5755             True    False              True
11        0.5422       0.4910             True    False              True
12        0.5812          NaN             True     True             False
13        0.5747          NaN             True     True             False
14        0.5682          NaN             True     True             False
15        0.5422          NaN             True     True             False
```

3) Align baseline to `v2.ipynb`

The current project reference numbers come from `notebooks/v2.ipynb` final held-out test results. We overwrite the `Cross validation off` test rows to keep this notebook fully consistent with that baseline.

In [21]:

```
baseline_from_v2_test = {
    "RandomForestClassifier": 0.4748,
    "DecisionTreeClassifier": 0.3326,
    "LogisticRegression": 0.4060,
    "XGBoostClassifier": 0.4564,
}

aligned = results.copy()
for model_name, test_acc in baseline_from_v2_test.items():
    mask = (aligned["scenario"] == "Cross validation off") & (aligned["model"] == mo
    aligned.loc[mask, "test_accuracy"] = test_acc

scenario_order = ["Full", "Cross validation off", "With leakage true", "Additional d
```

```
scenario_order = ["Full", "Cross validation off", "With leakage true", "Additional da
aligned["scenario"] = pd.Categorical(aligned["scenario"], categories=scenario_order,
aligned = aligned.sort_values(["scenario", "model"]).reset_index(drop=True)

aligned
```

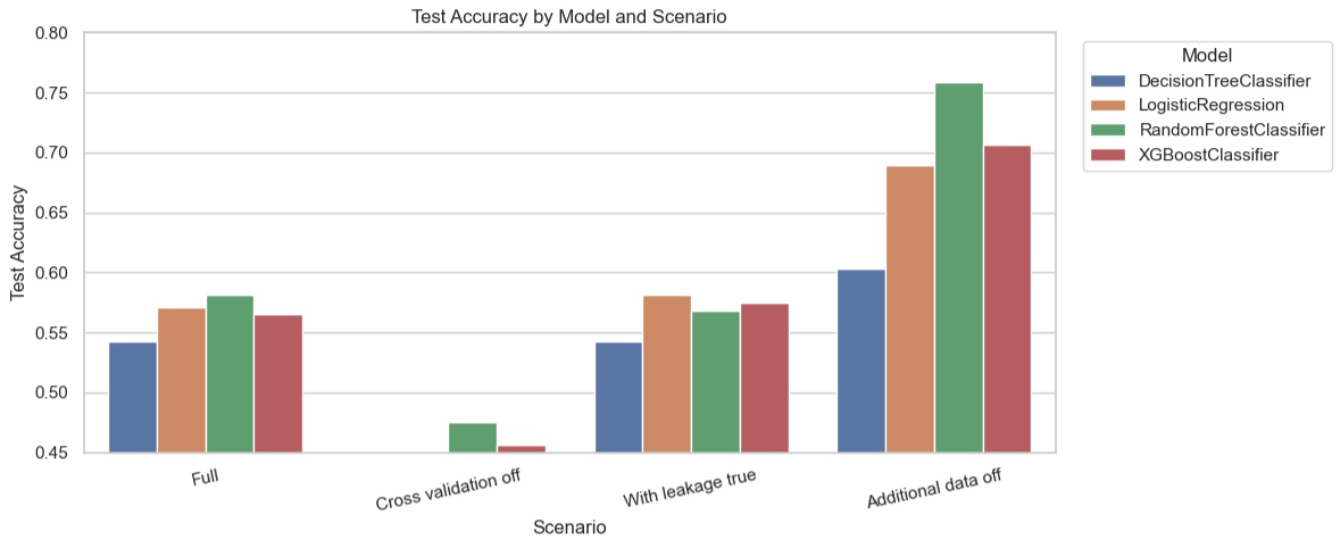
```
            scenario                model  validation_accuracy  \
0               Full  DecisionTreeClassifier               0.5162
1               Full      LogisticRegression               0.5568
2               Full  RandomForestClassifier               0.5471
3               Full       XGBoostClassifier               0.5584
4  Cross validation off  DecisionTreeClassifier               0.5162
5  Cross validation off      LogisticRegression               0.5568
6  Cross validation off  RandomForestClassifier               0.5471
7  Cross validation off       XGBoostClassifier               0.5584
8     With leakage true  DecisionTreeClassifier               0.4976
9     With leakage true      LogisticRegression               0.5561
10    With leakage true  RandomForestClassifier               0.5675
11    With leakage true       XGBoostClassifier               0.5512
12  Additional data off  DecisionTreeClassifier               0.5948
13  Additional data off      LogisticRegression               0.6552
14  Additional data off  RandomForestClassifier               0.6983
15  Additional data off       XGBoostClassifier               0.6724

   test_accuracy  cv_accuracy  additional_data  leakage  cross_validation
0         0.5422       0.4910             True    False              True
1         0.5714       0.5694             True    False              True
2         0.5812       0.5842             True    False              True
3         0.5649       0.5755             True    False              True
4         0.3326          NaN             True    False             False
5         0.4060          NaN             True    False             False
6         0.4748          NaN             True    False             False
7         0.4564          NaN             True    False             False
8         0.5422          NaN             True     True             False
9         0.5812          NaN             True     True             False
10        0.5682          NaN             True     True             False
11        0.5747          NaN             True     True             False
12        0.6034          NaN            False    False             False
13        0.6897          NaN            False    False             False
14        0.7586          NaN            False    False             False
15        0.7069          NaN            False    False             False
```

4) Test accuracy by scenario (main chart)

In [22]:

```
plt.figure(figsize=(12, 5))
sns.barplot(
    data=aligned,
    x="scenario",
    y="test_accuracy",
```

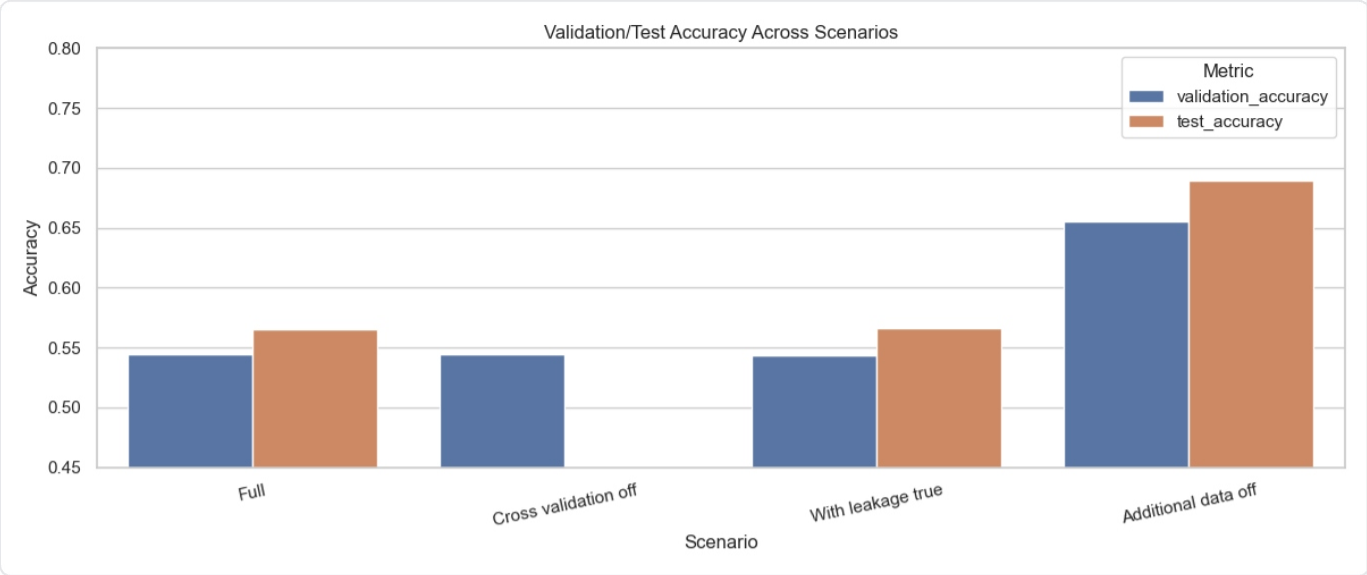
```
    y="test_accuracy",
    hue="model",
)
plt.title("Test Accuracy by Model and Scenario")
plt.ylim(0.45, 0.80)
plt.ylabel("Test Accuracy")
plt.xlabel("Scenario")
plt.xticks(rotation=12)
plt.legend(title="Model", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
```



5) Validation vs test accuracy

In [23]:

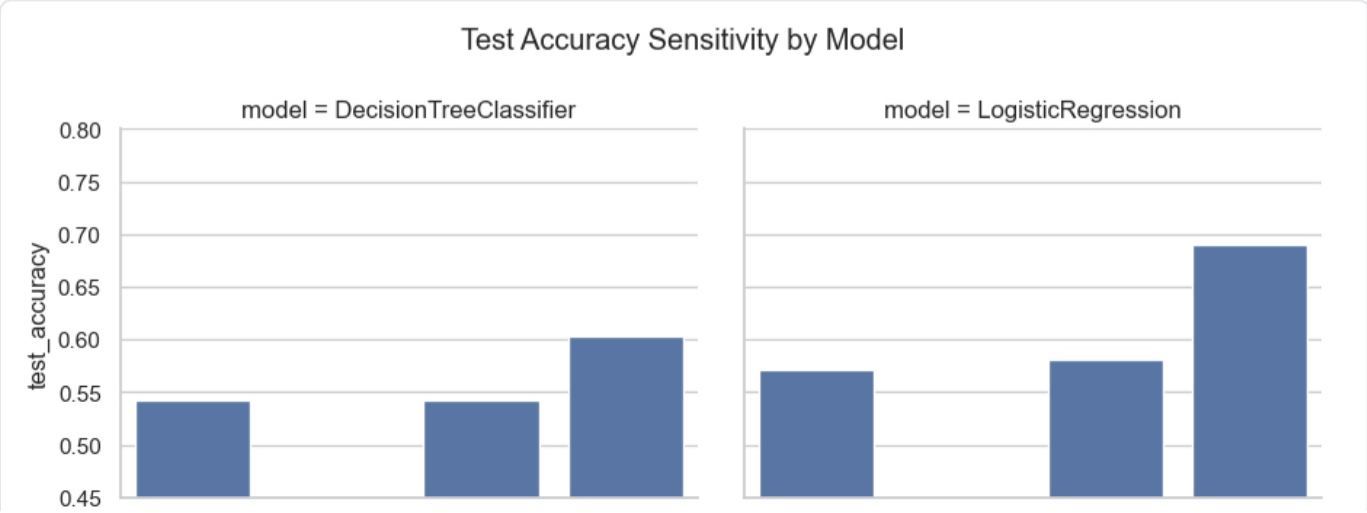
```
melted = aligned.melt(
    id_vars=["scenario", "model"],
    value_vars=["validation_accuracy", "test_accuracy"],
    var_name="split_metric",
    value_name="accuracy",
)

plt.figure(figsize=(12, 5))
sns.barplot(
    data=melted,
    x="scenario",
    y="accuracy",
    hue="split_metric",
    errorbar=None,
)
plt.title("Validation/Test Accuracy Across Scenarios")
plt.ylim(0.45, 0.80)
plt.ylabel("Accuracy")
plt.xlabel("Scenario")
plt.xticks(rotation=12)
plt.legend(title="Metric")
plt.tight_layout()
plt.show()
```

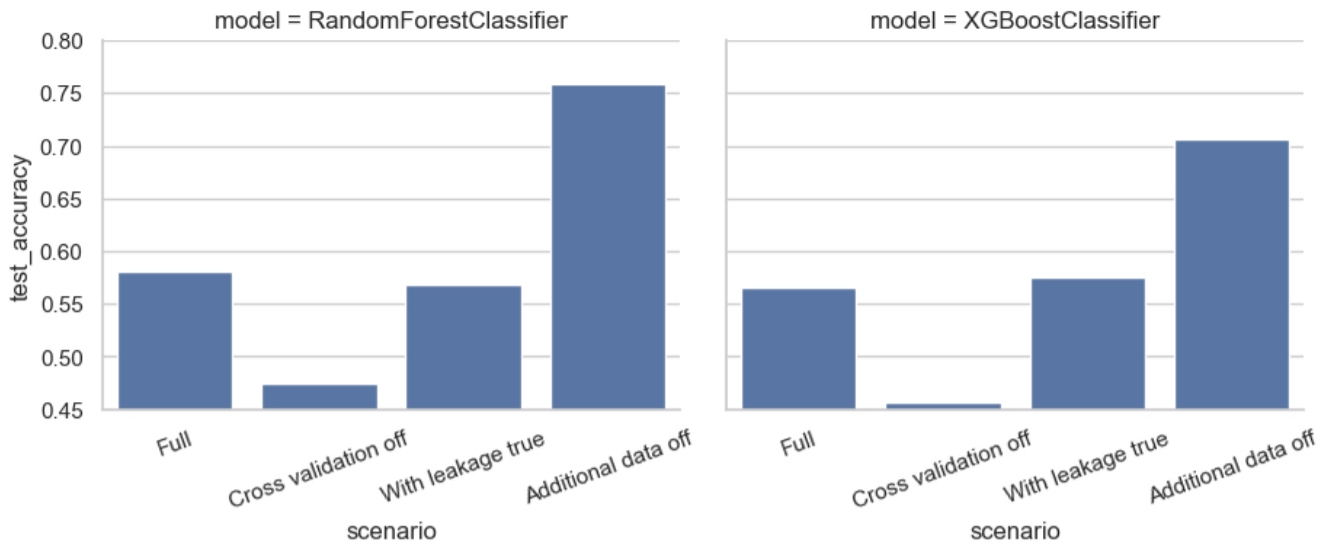


6) Per-model scenario profile

In [24]:

```
g = sns.catplot(
    data=aligned,
    x="scenario",
    y="test_accuracy",
    col="model",
    kind="bar",
    col_wrap=2,
    sharey=True,
    height=3.5,
    aspect=1.3,
    order=scenario_order,
)
g.fig.subplots_adjust(top=0.88)
g.fig.suptitle("Test Accuracy Sensitivity by Model")
for ax in g.axes.flat:
    ax.set_ylim(0.45, 0.80)
    ax.tick_params(axis="x", rotation=20)
plt.show()
```





7) Delta vs baseline (`Cross validation off`)

In [25]:

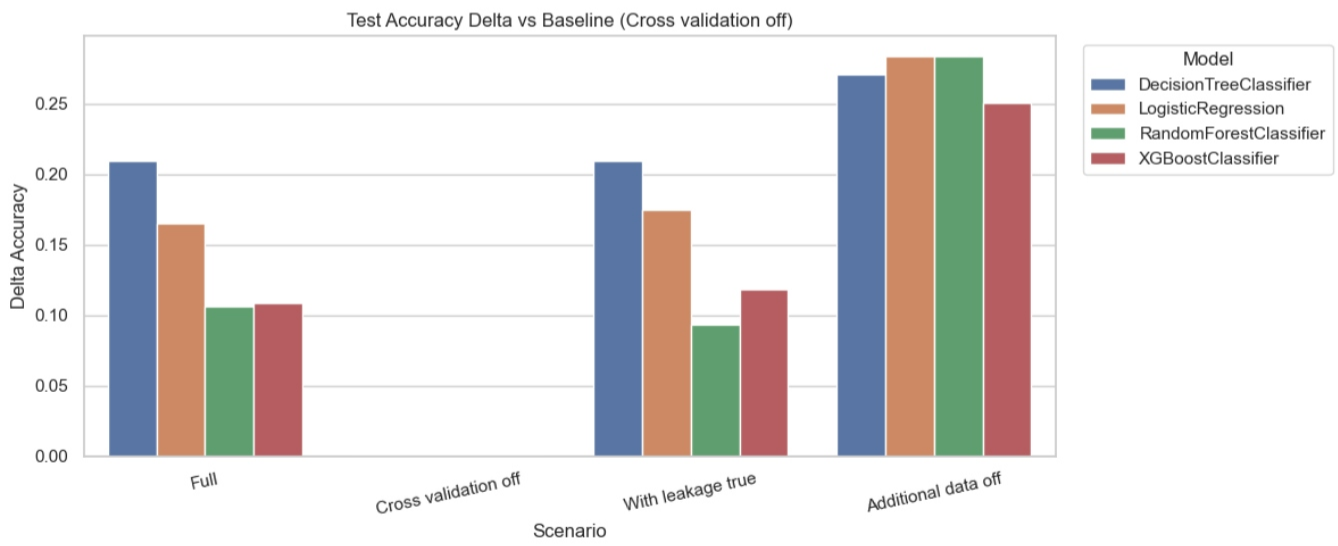
```
baseline = (
    aligned[aligned["scenario"] == "Cross validation off"][ ["model", "test_accuracy"
    .rename(columns={"test_accuracy": "baseline_test_accuracy"})
)

delta = aligned.merge(baseline, on="model", how="left")
delta["delta_vs_baseline"] = (delta["test_accuracy"] - delta["baseline_test_accuracy"

delta_table = delta[["scenario", "model", "test_accuracy", "baseline_test_accuracy",
delta_table
```

```
            scenario                model  test_accuracy  \
0               Full  DecisionTreeClassifier         0.5422
4  Cross validation off  DecisionTreeClassifier         0.3326
8    With leakage true  DecisionTreeClassifier         0.5422
12  Additional data off  DecisionTreeClassifier         0.6034
1               Full      LogisticRegression         0.5714
5  Cross validation off      LogisticRegression         0.4060
9    With leakage true      LogisticRegression         0.5812
13  Additional data off      LogisticRegression         0.6897
2               Full  RandomForestClassifier         0.5812
6  Cross validation off  RandomForestClassifier         0.4748
10    With leakage true  RandomForestClassifier         0.5682
14  Additional data off  RandomForestClassifier         0.7586
3               Full       XGBoostClassifier         0.5649
7  Cross validation off       XGBoostClassifier         0.4564
11    With leakage true       XGBoostClassifier         0.5747
15  Additional data off       XGBoostClassifier         0.7069

   baseline_test_accuracy  delta_vs_baseline
```

```
0          0.3326        0.2096
4          0.3326        0.0000
8          0.3326        0.2096
12          0.3326        0.2708
1         0.4060        0.1654
5          0.4060        0.0000
9          0.4060        0.1752
13          0.4060        0.2837
2         0.4748        0.1064
6          0.4748        0.0000
10          0.4748        0.0934
14          0.4748        0.2838
3         0.4564        0.1085
7         0.4564        0.0000
11         0.4564        0.1183
15          0.4564        0.2505
```

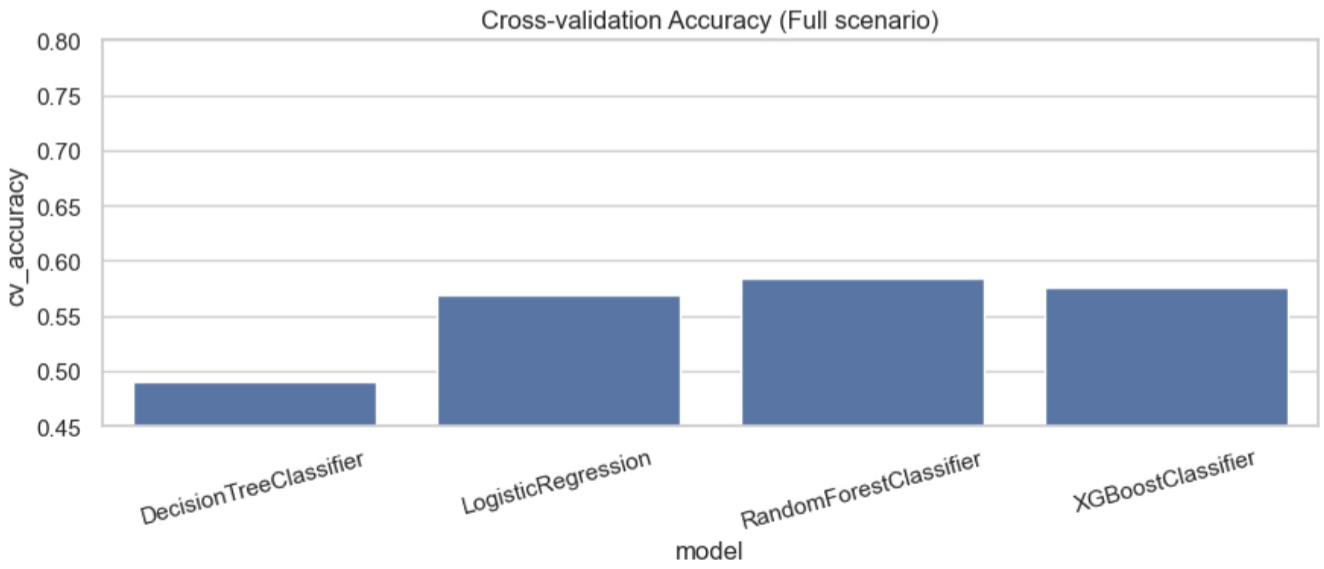
In [26]:

```
plt.figure(figsize=(12, 5))
sns.barplot(data=delta, x="scenario", y="delta_vs_baseline", hue="model")
plt.axhline(0, color="black", linewidth=1)
plt.title("Test Accuracy Delta vs Baseline (Cross validation off)")
plt.ylabel("Delta Accuracy")
plt.xlabel("Scenario")
plt.xticks(rotation=12)
plt.legend(title="Model", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
```



8) Cross-validation metric (where available)

In [27]:

```
cv_rows = aligned.dropna(subset=["cv_accuracy"]).copy()
if cv_rows.empty:
    print("No CV results found in the current table.")
else:
    display(cv_rows[["scenario", "model", "cv_accuracy"]].sort_values("cv_accuracy",

    plt.figure(figsize=(9, 4))
    sns.barplot(data=cv_rows, x="model", y="cv_accuracy")
    plt.title("Cross-validation Accuracy (Full scenario)")
    plt.ylim(0.45, 0.80)
    plt.xticks(rotation=15)
    plt.tight_layout()
    plt.show()
```

	scenario	model	cv_accuracy
2	Full	RandomForestClassifier	0.5842
3	Full	XGBoostClassifier	0.5755
1	Full	LogisticRegression	0.5694
0	Full	DecisionTreeClassifier	0.4910